



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验设计报告

开课学期: 2025 年春季
课程名称: 计算机系统
实验名称: Lab2 TinyShell
实验性质: 课内实验
实验时间: 5.14 地点: T2617
学生班级: 5
学生学号: 220110515
学生姓名: 金正达
评阅教师:
报告成绩:

实验与创新实践教育中心印制

2025 年 4 月

1. 需求分析

(列出 TinyShell 应支持的基本功能 (如执行外部命令、内置命令、命令行解析等))

1. 命令行解析

- 通过 `parseline` 函数解析用户输入的命令行
- 支持参数分割, 将命令行拆分为命令和参数
- 识别后台任务 (以 `&` 结尾的命令)

2. 执行外部命令

- 通过 `fork` 和 `execve` 执行非内置的外部命令
- 为子进程创建独立的进程组 (`setpgid`)
- 区分前台和后台作业的执行模式

3. 内置命令支持

- `quit`: 退出 shell
- `jobs`: 显示当前作业列表
- `bg <job>`: 将停止的作业放到后台继续运行
- `fg <job>`: 将后台或停止的作业调到前台运行

4. 作业控制

- 维护作业列表 (通过 `jobs` 数组)
- 支持作业状态: 前台(FG)、后台(BG)、停止(ST)
- 跟踪进程状态变化 (创建、终止、停止)

5. 信号处理

- `SIGINT` (`Ctrl+C`): 终止前台作业
- `SIGTSTP` (`Ctrl+Z`): 停止前台作业
- `SIGCHLD`: 处理子进程状态变化 (终止或停止)
- `SIGQUIT`: 优雅地终止 shell

6. 前台作业管理

- 等待前台作业完成 (`waitfg` 函数)
- 允许用户将前台作业移至后台或停止前台作业

7. 后台作业管理

- 允许作业在后台执行
- 通过 `jobs` 命令查看后台作业
- 支持后台作业状态变化通知

8. 其他辅助功能

- 作业列表管理 (添加、删除、查找作业)
- 进程 ID 和作业 ID 之间的转换
- 错误处理和用户提示

2. 设计思路

(设计 TinyShell 的整体架构, 包括主函数、命令行解析函数、内置命令处理函数等。确

定函数之间的调用关系和参数传递方式。)

主控循环

Shell 的核心是在 main 函数中实现的"读取-解析-执行"循环:

- 1) 初始化 (信号处理、作业列表)
- 2) 循环处理用户输入
 - a) 读取命令行 (fgets)
 - b) 评估执行命令 (eval)
- 3) 终止

命令执行流程

命令执行由 eval 函数处理, 流程为:

- 1) 调用 parseline 解析用户输入, 得到参数数组和后台标志
- 2) 检查是否为内置命令 (builtin_cmd)
- 3) 如果是内置命令, 直接执行
- 4) 如果是外部命令, 创建子进程执行:
 - i. 阻塞 SIGCHLD 信号, 避免竞态条件
 - ii. 创建子进程 (fork)
 - iii. 在子进程中设置独立进程组并执行程序 (execve)
 - iv. 在父进程中添加作业到作业列表
 - v. 根据前台/后台标志决定等待还是继续

作业控制系统

使用 job_t 结构体数组维护所有作业状态:

- 每个作业记录包含 PID、JID、状态和命令行
- 支持三种状态: 前台(FG)、后台(BG)、停止(ST)
- 提供添加、删除、查找和列出作业的接口函数

信号处理子系统

三个关键信号处理函数:

- sigchld_handler: 处理子进程状态变化 (终止或停止)
- sigint_handler: 处理 Ctrl+C, 终止前台作业
- sigtstp_handler: 处理 Ctrl+Z, 停止前台作业

函数调用关系

主要函数调用层次结构:

main: Shell 入口点

- 1) 安装信号处理器
- 2) 初始化作业列表
- 3) 启动命令循环

eval: 命令执行入口

- 1) 调用 parseline 解析命令
- 2) 调用 builtin_cmd 检查内置命令

- 3) 处理外部命令 (fork/exec)
- 4) 调用 addjob 添加作业
- 5) 调用 waitfg 等待前台作业

内置命令处理

- builtin_cmd: 识别内置命令
- do_bgfg: 处理 bg/fg 命令
- listjobs: 列出所有作业

信号处理

- sigchld_handler: 处理子进程终止/停止
- sigint_handler: 处理 Ctrl+C
- sigtstp_handler: 处理 Ctrl+Z

作业控制辅助函数

- addjob/deletejob: 管理作业列表
- getjobpid/getjobjid: 查找作业
- fgpids: 获取前台作业

参数传递方式

- 1) 命令行解析:
 - a) parseline 接收原始命令行字符串
 - b) 返回参数数组和后台标志 (通过参数引用和返回值)
- 2) 作业状态传递:
 - a) 通过全局 jobs 数组在不同函数间共享作业信息
 - b) 使用 PID 和 JID 作为作业标识符
- 3) 信号处理:
 - a) 信号处理函数通过信号值参数接收信号
 - b) 访问全局 jobs 数组更新作业状态
- 4) 进程同步:
 - a) 使用信号集和 sigprocmask 阻塞信号, 防止竞态条件
 - b) 使用 sigsuspend 在等待前台作业时挂起

并发处理策略

- 1) 信号阻塞:
 - a) 在关键部分 (如添加/删除作业) 阻塞信号
 - b) 避免信号处理器与主程序之间的竞态条件
- 2) 进程组管理:
 - a) 为每个作业创建独立进程组
 - b) 确保信号只发送给正确的目标进程
- 3) 状态跟踪:
 - a) 通过 SIGCHLD 处理器跟踪子进程状态变化
 - b) 维护作业列表的一致性

3. 核心函数实现

(深入分析核心函数的实现)

3.1 void eval(char *cmdline)函数

函数功能:

eval 函数是 shell 的核心函数，负责解析并执行用户输入的命令行。它首先判断命令是否为内置命令（如 quit、jobs、bg、fg），如果是则立即执行；否则创建子进程来执行外部程序，并根据命令是前台还是后台作业来相应处理。

参数:

cmdline: 用户输入的命令行字符串

处理流程:

- 1) 解析命令行，获取参数列表和前/后台运行标志
- 2) 忽略空命令行
- 3) 检查是否为内置命令，若是则直接执行
- 4) 若非内置命令，则创建子进程执行外部程序:
 - a) 阻塞 SIGCHLD 信号，防止竞争条件
 - b) 调用 fork() 创建子进程
 - c) 子进程设置独立进程组并执行程序
 - d) 父进程将作业添加到作业列表
 - e) 根据前台/后台标志决定父进程行为

要点分析:

- 1) 信号处理：使用信号阻塞（sigprocmask）防止在添加作业到作业列表前子进程先终止而导致的竞争条件
- 2) 进程组管理：通过 setpgid(0, 0) 将子进程设为新进程组的组长，确保子进程能独立接收信号，避免前台进程组中的其他进程受到影响
- 3) 作业控制：根据命令是前台还是后台运行，采取不同处理策略：
 - a) 前台作业：父进程等待子进程结束（调用 waitfg）
 - b) 后台作业：父进程不等待，打印作业信息后继续
- 4) 错误处理：子进程执行失败时打印错误信息并退出
- 5) 同步机制：通过阻塞/解除阻塞信号实现父子进程间的同步，确保作业管理的正确性

3.2 int builtin_cmd(char **argv)函数

函数功能:

builtin_cmd 函数检查用户输入的命令是否为内置命令（built-in command），如果是则立即执行该命令并返回 1（真），如果不是则返回 0（假）。内置命令包括 "quit"、"jobs"、"bg" 和 "fg"。

参数:

argv: 命令行参数数组，其中 argv[0] 是命令名称，后续元素是命令的参数

处理流程:

- 1) 检查命令名称（argv[0]）是否是 "quit"，如果是则直接退出 shell
- 2) 检查命令名称是否是 "jobs"，如果是则列出所有作业并返回 1
- 3) 检查命令名称是否是 "bg" 或 "fg"，如果是则调用 do_bgfg 函数处理并返回 1

- 4) 如果命令不是以上任何一个内置命令，则返回 0，表示这是一个外部命令

要点分析：

- 1) 简单高效的实现：通过一系列的字符串比较（strcmp）确定命令类型，实现清晰直观
- 2) 返回值的意义：
 - a) 返回 1 表示命令已被识别并执行，调用者（eval 函数）不需要再处理
 - b) 返回 0 表示不是内置命令，调用者需要创建子进程来执行外部命令
- 3) 命令处理方式：
 - a) "quit" 命令直接调用 exit(0) 退出整个 shell
 - b) "jobs" 命令调用 listjobs(jobs) 显示当前作业列表
 - c) "bg"/"fg" 命令通过调用 do_bgfg 函数进行更复杂的处理
- 4) 模块化设计：将前台/后台作业控制的复杂逻辑封装在 do_bgfg 函数中，保持了 builtin_cmd 的简洁

3.3 void do_bgfg(char **argv) 函数

函数功能：

do_bgfg 函数实现 shell 的内置命令 bg 和 fg，用于控制作业的前台/后台运行状态。它允许用户将停止的作业恢复执行，或者将后台作业调到前台运行，或将前台作业放到后台运行。

参数：

argv: 命令行参数数组，其中 argv[0] 是命令名称（"bg" 或 "fg"），argv[1] 是作业标识符（PID 或以 % 开头的作业 ID）

处理流程：

- 1) 检查是否提供了作业标识符参数，若无则打印错误信息并返回
- 2) 分析作业标识符类型（JID 或 PID）：
 - a) 如果以 '%' 开头，则按作业 ID（JID）查找
 - b) 否则按进程 ID（PID）查找
- 3) 检查作业标识符的有效性，若无效则打印错误信息并返回
- 4) 根据标识符查找对应的作业，若不存在则打印错误信息并返回
- 5) 向作业所在的进程组发送 SIGCONT 信号，使其继续执行
- 6) 根据命令类型（bg 或 fg）更新作业状态：
 - a) 如果是 bg 命令，将作业状态设为后台运行（BG）并打印作业信息
 - b) 如果是 fg 命令，将作业状态设为前台运行（FG）并等待作业完成

要点分析：

- 1) 错误处理完善：函数对各种可能的错误情况都进行了检查和处理，包括：
 - a) 缺少参数时的提示
 - b) 参数格式不正确时的提示
 - c) 作业不存在时的提示
- 2) 作业标识符的双重支持：
 - a) 支持以 PID 形式指定作业（如 fg 1234）
 - b) 支持以 JID 形式指定作业（如 fg %1）
- 3) 进程组信号控制：
 - a) 使用 kill(-job->pid, SIGCONT) 向整个进程组发送信号
 - b) 负号表示发送给进程组而不是单个进程，确保所有相关进程都能接收到信号
- 4) 状态转换管理：

- a) 明确地更新作业的状态（BG 或 FG）
- b) 对于前台作业，使用 `waitfg` 函数等待其完成，实现了前台作业的阻塞特性

3.4 `void waitfg(pid_t pid)` 函数

函数功能：

`waitfg` 函数使 shell 暂停执行，直到指定的前台进程不再是前台进程为止。这个函数实现了 shell 等待前台作业完成的机制，避免 shell 立即返回到命令提示符。

参数：

`pid`: 前台进程的进程 ID，是 shell 需要等待的进程

处理流程：

- 1) 创建并初始化一个空的信号集
- 2) 使用循环检查指定进程是否仍是前台进程：
 - a) 调用 `fgpid(jobs)` 获取当前前台进程 ID
 - b) 当获取的 ID 与参数 `pid` 匹配时，表示进程仍在前台运行
- 3) 在循环内部使用 `sigsuspend(&mask)` 暂停 shell，等待信号到达
- 4) 当前台进程状态改变（终止、停止或被移至后台）时，shell 被唤醒并检查条件
- 5) 当 `pid` 不再是前台进程时，退出循环并返回

要点分析：

- 1) 使用 `sigsuspend` 的意义：
 - a) 采用 `sigsuspend` 而非忙等待，避免 CPU 资源浪费
 - b) 仅当有信号到达时才被唤醒，提高效率
 - c) 相比 `sleep` 更精确，不会有固定延迟
- 2) 循环检查的必要性：
 - a) 防止"虚假唤醒"问题，即使被唤醒也需要再次检查条件
 - b) 确保只有在前台进程真正结束或状态改变时才返回
- 3) 信号集的初始化：
 - a) 使用 `sigemptyset` 创建空信号集
 - b) 暂停期间允许接收所有信号（包括 `SIGCHLD`、`SIGINT`、`SIGTSTP`）
- 4) 避免竞争条件：
 - a) 该函数适当处理了父子进程之间可能的竞争条件
 - b) 确保 shell 只有在子进程确实不再是前台进程后才继续执行
- 5) 与 `SIGCHLD` 处理器配合：
 - a) 此函数与 `sigchld_handler` 密切配合
 - b) 当子进程状态改变时，`SIGCHLD` 处理器更新作业列表，同时唤醒在 `sigsuspend` 中等待的 shell

3.5 `void sigchld_handler(int sig)` 函数

函数功能：

`sigchld_handler` 函数是 `SIGCHLD` 信号的处理函数，负责处理子进程状态变化的事件。当子进程终止（成为僵尸进程）或因接收到 `SIGSTOP`/`SIGTSTP` 信号而停止时，内核会向父进程（shell）

发送 SIGCHLD 信号。该函数会对这些子进程进行回收 (reap)，更新作业列表，并在适当情况下向用户显示作业状态信息。

参数：

sig: 信号编号，在本函数中为 SIGCHLD (子进程状态变化信号)

处理流程：

- 1) 保存当前的 errno 值，避免处理程序修改它
- 2) 使用 waitpid 函数循环检查并处理所有已终止或停止的子进程：
 - a) 指定参数 -1 表示等待任何子进程
 - b) 使用 WNOHANG 标志使得函数立即返回，不会阻塞
 - c) 使用 WUNTRACED 标志检测停止的子进程
- 3) 根据子进程的状态执行不同的处理：
 - a) 如果子进程正常终止 (WIFEXITED)，则从作业列表中删除该作业
 - b) 如果子进程被信号终止 (WIFSIGNALED)，则打印终止信息并删除该作业
 - c) 如果子进程被信号停止 (WIFSTOPPED)，则打印停止信息并将作业状态更新为停止 (ST)
- 4) 循环结束后恢复之前保存的 errno 值

要点分析：

- 1) 错误处理与恢复：
 - a) 使用 olderrno 保存和恢复 errno，确保信号处理不会干扰程序正常的错误处理流程
- 2) 非阻塞式循环处理：
 - a) 使用 waitpid 函数的 WNOHANG 标志实现非阻塞调用
 - b) 使用 while 循环处理所有已终止的子进程，避免丢失信号
 - c) 这种设计应对了 SIGCHLD 信号可能合并的情况 (多个子进程几乎同时终止)
- 3) 全面的状态检测：
 - a) 使用 WIFEXITED、WIFSIGNALED 和 WIFSTOPPED 宏全面检测子进程的不同终止或停止状态
 - b) 通过 WTERMSIG 和 WSTOPSIG 宏获取导致终止或停止的具体信号
- 4) 用户友好的信息输出：
 - a) 当子进程因信号终止或停止时，打印信息告知用户
 - b) 使用 pid2jid 将进程 ID 转换为作业 ID，使信息更加友好
- 5) 作业状态管理：
 - a) 正确更新作业状态 (删除已终止作业，将停止的作业状态改为 ST)

4. 自测试结果

(请提供测试用例 trace01 至 trace16 的自测试输出截图，并与 tshref 的测试结果进行对比分析，确保结果的一致性；若存在差异，请明确指出不同之处。)

4.1 测试用例 trace01.txt 的输出截图

测试结论：通过测试

4.2 测试用例 trace02.txt 的输出截图

```
./sdriver.pl -t trace02.txt -s ./tsh -a "-p"
#
# trace02.txt - Process builtin quit command.
#
6 ./sdriver.pl -t trace02.txt -s ./tsh -a "-p"
7 #
8 # trace02.txt - Process builtin quit command.
9 #
```

测试结论：通过测试

4.3 测试用例 trace03.txt 的输出截图

```
./sdriver.pl -t trace03.txt -s ./tsh -a "-p"
#
# trace03.txt - Run a foreground job.
#
tsh> quit
10 ./sdriver.pl -t trace03.txt -s ./tsh -a "-p"
11 #
12 # trace03.txt - Run a foreground job.
13 #
14 tsh> quit
```

测试结论：通过测试

4.4 测试用例 trace04.txt 的输出截图

```
./sdriver.pl -t trace04.txt -s ./tsh -a "-p"
#
# trace04.txt - Run a background job.
#
tsh> ./myspin 1 &
[1] (19237) ./myspin 1 &
15 ./sdriver.pl -t trace04.txt -s ./tsh -a "-p"
16 #
17 # trace04.txt - Run a background job.
18 #
19 tsh> ./myspin 1 &
20+ [1] (26252) ./myspin 1 &
```

测试结论：通过测试

4.5 测试用例 trace05.txt 的输出截图

```
./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
#
# trace05.txt - Process jobs builtin command.
#
tsh> ./myspin 2 &
[1] (19260) ./myspin 2 &
tsh> ./myspin 3 &
[2] (19262) ./myspin 3 &
tsh> jobs
[1] (19260) Running ./myspin 2 &
[2] (19262) Running ./myspin 3 &
21 ./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
22 #
23 # trace05.txt - Process jobs builtin command.
24 #
25 tsh> ./myspin 2 &
26+ [1] (26256) ./myspin 2 &
27 tsh> ./myspin 3 &
28+ [2] (26258) ./myspin 3 &
29 tsh> jobs
30+ [1] (26256) Running ./myspin 2 &
31+ [2] (26258) Running ./myspin 3 &
```

测试结论：通过测试

4.6 测试用例 trace06.txt 的输出截图

```
./sdriver.pl -t trace06.txt -s ./tsh -a "-p"
#
# trace06.txt - Forward SIGINT to foreground job.
#
tsh> ./myspin 4
Job [1] (19303) terminated by signal 2
32 ./sdriver.pl -t trace06.txt -s ./tsh -a "-p"
33 #
34 # trace06.txt - Forward SIGINT to foreground job.
35 #
36 tsh> ./myspin 4
37+ Job [1] (26263) terminated by signal 2
```

测试结论：通过测试

4.7 测试用例 trace07.txt 的输出截图

```

./sdriver.pl -t trace07.txt -s ./tsh -a "-p"
#
# trace07.txt - Forward SIGINT only to foreground job.
#
tsh> ./myspin 4 &
[1] (19308) ./myspin 4 &
tsh> ./myspin 5
Job [2] (19310) terminated by signal 2
tsh> jobs
[1] (19308) Running ./myspin 4 &
38 ./sdriver.pl -t trace07.txt -s ./tsh -a "-p"
39 #
40 # trace07.txt - Forward SIGINT only to foreground job.
41 #
42 tsh> ./myspin 4 &
43+ [1] (26267) ./myspin 4 &
44 tsh> ./myspin 5
45+ Job [2] (26269) terminated by signal 2
46 tsh> jobs
47+ [1] (26267) Running ./myspin 4 &

```

测试结论：通过测试

4.8 测试用例 trace08.txt 的输出截图

```

./sdriver.pl -t trace08.txt -s ./tsh -a "-p"
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
tsh> ./myspin 4 &
[1] (19349) ./myspin 4 &
tsh> ./myspin 5
Job [2] (19351) stopped by signal 20
tsh> jobs
[1] (19349) Running ./myspin 4 &
[2] (19351) Stopped ./myspin 5
48 ./sdriver.pl -t trace08.txt -s ./tsh -a "-p"
49 #
50 # trace08.txt - Forward SIGTSTP only to foreground job.
51 #
52 tsh> ./myspin 4 &
53+ [1] (26274) ./myspin 4 &
54 tsh> ./myspin 5
55+ Job [2] (26276) stopped by signal 20
56 tsh> jobs
57+ [1] (26274) Running ./myspin 4 &
58+ [2] (26276) Stopped ./myspin 5

```

测试结论：通过测试

4.9 测试用例 trace09.txt 的输出截图

```

./sdriver.pl -t trace09.txt -s ./tsh -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (19395) ./myspin 4 &
tsh> ./myspin 5
Job [2] (19397) stopped by signal 20
tsh> jobs
[1] (19395) Running ./myspin 4 &
[2] (19397) Stopped ./myspin 5
tsh> bg %2
[2] (19397) ./myspin 5
tsh> jobs
[1] (19395) Running ./myspin 4 &
[2] (19397) Running ./myspin 5
59 ./sdriver.pl -t trace09.txt -s ./tsh -a "-p"
60 #
61 # trace09.txt - Process bg builtin command
62 #
63 tsh> ./myspin 4 &
64+ [1] (26281) ./myspin 4 &
65 tsh> ./myspin 5
66+ Job [2] (26283) stopped by signal 20
67 tsh> jobs
68+ [1] (26281) Running ./myspin 4 &
69+ [2] (26283) Stopped ./myspin 5
70 tsh> bg %2
71+ [2] (26283) ./myspin 5
72 tsh> jobs
73+ [1] (26281) Running ./myspin 4 &
74+ [2] (26283) Running ./myspin 5

```

测试结论：通过测试

4.10 测试用例 trace10.txt 的输出截图

```

./sdriver.pl -t trace10.txt -s ./tsh -a "-p"
#
# trace10.txt - Process fg builtin command.
#
tsh> ./myspin 4 &
[1] (19438) ./myspin 4 &
tsh> fg %1
Job [1] (19438) stopped by signal 20
tsh> jobs
[1] (19438) Stopped ./myspin 4 &
tsh> fg %1
tsh> jobs
75 ./sdriver.pl -t trace10.txt -s ./tsh -a "-p"
76 #
77 # trace10.txt - Process fg builtin command.
78 #
79 tsh> ./myspin 4 &
80+ [1] (26290) ./myspin 4 &
81 tsh> fg %1
82+ Job [1] (26290) stopped by signal 20
83 tsh> jobs
84+ [1] (26290) Stopped ./myspin 4 &
85 tsh> fg %1
86 tsh> jobs

```

测试结论：通过测试

4.11 测试用例 trace11.txt 的输出截图

```

./sdriver.pl -t trace11.txt -s ./tsh -a "-p"
#
# trace11.txt - Forward SIGINT to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (19477) terminated by signal 2
tsh> /bin/ps a
  PID TTY          STAT TIME  COMMAND
 267 hvc0      Ss+   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud
 275 tty1      Ss+   0:00 /sbin/agetty -o -p -- \u --noclear - linux
 362 pts/1     Ss    0:00 /bin/login -f
 383 pts/0     Ss+   0:00 sh -c "$VSCODE_WSL_EXT_LOCATION/scripts/wslServ
 384 pts/0     S+    0:00 sh /mnt/c/Users/Jin_zd/.vscode/extensions/ms-vs
 435 pts/1     S+    0:00 -bash
 584 pts/0     S+    0:00 sh /home/jin_zd/.vscode-server/bin/91fa95bccb027e
 588 pts/0     SL+   0:17 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
 766 pts/0     SL+   0:15 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1475 pts/2     Ssl+  0:02 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1483 pts/0     SL+   0:02 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1490 pts/3     Ssl+  0:08 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1505 pts/0     SL+   6:05 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1561 pts/4     Ss+   0:00 /bin/sh -c cd '/home/jin_zd/csapp/lab2/shlab-hu
1562 pts/4     S+    0:00 /bin/sh
1572 pts/4     SL+   0:00 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1605 pts/4     S+    0:00 /bin/sh
1661 pts/0     SL+   0:33 /home/jin_zd/.vscode-server/extensions/ms-vsco
1742 pts/0     SL+   0:11 /home/jin_zd/.vscode-server/extensions/ms-vsco
19100 pts/5    Ss    0:00 /bin/bash --init-file /home/jin_zd/.vscode-serv
19222 pts/5    S+    0:00 /bin/bash ./tsh_output.sh
19474 pts/5    S+    0:00 /usr/bin/perl ./sdriver.pl -t trace11.txt -s ./
19475 pts/5    S+    0:00 ./tsh -p
19486 pts/5    R     0:00 /bin/ps a
87 ./sdriver.pl -t trace11.txt -s ./tsh -a "-p"
88 #
89 # trace11.txt - Forward SIGINT to every process in foreground process g
90 #
91 tsh> ./mysplit 4
92+ Job [1] (26298) terminated by signal 2
93 tsh> /bin/ps a
  PID TTY          STAT TIME  COMMAND
95+ 25181 pts/3    S     0:00 -usr/local/bin/tcsh -i
96+ 26239 pts/3    S     0:00 make tshrefout
97+ 26240 pts/3    S     0:00 /bin/sh -c make tests > tshref.out 2>&1
98+ 26241 pts/3    S     0:00 make tests
99+ 26295 pts/3    S     0:00 perl ./sdriver.pl -t trace11.txt -s ./tsh -a
100+ 26296 pts/3    S     0:00 ./tsh -p
101+ 26301 pts/3    R     0:00 /bin/ps a

```

测试结论：通过测试

4.12 测试用例 trace12.txt 的输出截图

```

./sdriver.pl -t trace12.txt -s ./tsh -a "-p"
#
# trace12.txt - Forward SIGTSTP to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (19491) stopped by signal 20
tsh> jobs
[1] (19491) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY          STAT TIME  COMMAND
 267 hvc0      Ss+   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud
 275 tty1      Ss+   0:00 /sbin/agetty -o -p -- \u --noclear - linux
 362 pts/1     Ss    0:00 /bin/login -f
 383 pts/0     Ss+   0:00 sh -c "$VSCODE_WSL_EXT_LOCATION/scripts/wslServ
 384 pts/0     S+    0:00 sh /mnt/c/Users/Jin_zd/.vscode/extensions/ms-vs
 435 pts/1     S+    0:00 -bash
 584 pts/0     S+    0:00 sh /home/jin_zd/.vscode-server/bin/91fa95bccb027e
 588 pts/0     SL+   0:17 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
 766 pts/0     SL+   0:15 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1475 pts/2     Ssl+  0:02 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1483 pts/0     SL+   0:02 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1490 pts/3     Ssl+  0:08 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1505 pts/0     SL+   6:05 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1561 pts/4     Ss+   0:00 /bin/sh -c cd '/home/jin_zd/csapp/lab2/shlab-hu
1562 pts/4     S+    0:00 /bin/sh
1572 pts/4     SL+   0:00 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1605 pts/4     S+    0:00 /bin/sh
1661 pts/0     SL+   0:33 /home/jin_zd/.vscode-server/extensions/ms-vsco
1742 pts/0     SL+   0:11 /home/jin_zd/.vscode-server/extensions/ms-vsco
19100 pts/5    Ss    0:00 /bin/bash --init-file /home/jin_zd/.vscode-serv
19222 pts/5    S+    0:00 /bin/bash ./tsh_output.sh
19488 pts/5    S+    0:00 /usr/bin/perl ./sdriver.pl -t trace12.txt -s ./
19489 pts/5    S+    0:00 ./tsh -p
19491 pts/5    T     0:00 ./mysplit 4
19492 pts/5    T     0:00 ./mysplit 4
19522 pts/5    R     0:00 /bin/ps a
102 ./sdriver.pl -t trace12.txt -s ./tsh -a "-p"
103 #
104 # trace12.txt - Forward SIGTSTP to every process in foreground process
105 #
106 tsh> ./mysplit 4
107+ Job [1] (26305) stopped by signal 20
108 tsh> jobs
109+ [1] (26305) Stopped ./mysplit 4
110 tsh> /bin/ps a
  PID TTY          STAT TIME  COMMAND
112+ 25181 pts/3    S     0:00 -usr/local/bin/tcsh -i
113+ 26239 pts/3    S     0:00 make tshrefout
114+ 26240 pts/3    S     0:00 /bin/sh -c make tests > tshref.out 2>&1
115+ 26241 pts/3    S     0:00 make tests
116+ 26302 pts/3    S     0:00 perl ./sdriver.pl -t trace12.txt -s ./tsh -a
117+ 26303 pts/3    S     0:00 ./tsh -p
118+ 26305 pts/3    T     0:00 ./mysplit 4
119+ 26306 pts/3    T     0:00 ./mysplit 4
120+ 26309 pts/3    R     0:00 /bin/ps a

```

测试结论：通过测试

4.13 测试用例 trace13.txt 的输出截图

```

./sdriver.pl -t trace13.txt -s ./tsh -a "-p"
#
# trace13.txt - Restart every stopped process in process group
#
tsh> ./mysplit 4
Job [1] (19528) stopped by signal 20
tsh> jobs
[1] (19528) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY          STAT       TIME COMMAND
 267 hvc0      Ss+        0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud
 275 tty1      Ss+        0:00 /sbin/agetty -o -p -- \u --noclear - linux
 362 pts/1     Ss         0:00 /bin/login -f
 383 pts/0     Ss+        0:00 sh -c "$VSCODE_WSL_EXT_LOCATION/scripts/wslServ
 384 pts/0     S+         0:00 sh /mnt/c/Users/Jin_zd/.vscode/extensions/ms-vs
 435 pts/1     S+         0:00 -bash
 584 pts/0     S+         0:00 sh /home/jin_zd/.vscode-server/bin/91fa95bccb027e
 588 pts/0     Sl+        0:17 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
 766 pts/0     Sl+        0:15 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1475 pts/2     Ssl+       0:02 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1483 pts/0     Sl+        0:02 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1490 pts/3     Ssl+       0:08 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1505 pts/0     Sl+        6:05 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1561 pts/4     Ss+        0:00 /bin/sh -c cd '/home/jin_zd/csapp/lab2/shlab-h
1562 pts/4     S+         0:00 /bin/sh
1572 pts/4     Sl+        0:00 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1605 pts/4     S+         0:00 /bin/sh
1661 pts/0     Sl+        0:33 /home/jin_zd/.vscode-server/extensions/ms-vsco
1742 pts/0     Sl+        0:11 /home/jin_zd/.vscode-server/extensions/ms-vsco
19100 pts/5     Ss         0:00 /bin/bash --init-file /home/jin_zd/.vscode-ser
19222 pts/5     S+         0:00 /bin/bash ./tsh_output.sh
19524 pts/5     S+         0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s .
19525 pts/5     S+         0:00 ./tsh -p
19528 pts/5     T          0:00 ./mysplit 4
19531 pts/5     T          0:00 ./mysplit 4
19537 pts/5     R          0:00 /bin/ps a
tsh> fg %1

121 ./sdriver.pl -t trace13.txt -s ./tsh -a "-p"
122 #
123 # trace13.txt - Restart every stopped process in process group
124 #
125 tsh> ./mysplit 4
126+ Job [1] (26313) stopped by signal 20
127 tsh> jobs
128+ [1] (26313) Stopped ./mysplit 4
129 tsh> /bin/ps a
  PID TTY          STAT       TIME COMMAND
1314+ 25181 pts/3     S          0:00 -usr/local/bin/tcsh -i
1324+ 26239 pts/3     S          0:00 make tshrefout
1334+ 26240 pts/3     S          0:00 /bin/sh -c make tests > tshref.out 2>&1
1344+ 26241 pts/3     S          0:00 make tests

1354+ 26310 pts/3     S          0:00 perl ./sdriver.pl -t trace13.txt -s ./tsh -a
1364+ 26311 pts/3     S          0:00 ./tsh -p
1374+ 26313 pts/3     T          0:00 ./mysplit 4
1384+ 26314 pts/3     T          0:00 ./mysplit 4
1394+ 26317 pts/3     R          0:00 /bin/ps a
140 tsh> fg %1

tsh> fg %1
tsh> /bin/ps a
  PID TTY          STAT       TIME COMMAND
 267 hvc0      Ss+        0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud
 275 tty1      Ss+        0:00 /sbin/agetty -o -p -- \u --noclear - linux
 362 pts/1     Ss         0:00 /bin/login -f
 383 pts/0     Ss+        0:00 sh -c "$VSCODE_WSL_EXT_LOCATION/scripts/wslServ
 384 pts/0     S+         0:00 sh /mnt/c/Users/Jin_zd/.vscode/extensions/ms-vs
 435 pts/1     S+         0:00 -bash
 584 pts/0     S+         0:00 sh /home/jin_zd/.vscode-server/bin/91fa95bccb027e
 588 pts/0     Sl+        0:17 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
 766 pts/0     Sl+        0:15 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1475 pts/2     Ssl+       0:02 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1483 pts/0     Sl+        0:02 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1490 pts/3     Ssl+       0:08 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1505 pts/0     Sl+        6:05 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1561 pts/4     Ss+        0:00 /bin/sh -c cd '/home/jin_zd/csapp/lab2/shlab-h
1562 pts/4     S+         0:00 /bin/sh
1572 pts/4     Sl+        0:00 /home/jin_zd/.vscode-server/bin/91fa95bccb027e
1605 pts/4     S+         0:00 /bin/sh
1661 pts/0     Sl+        0:33 /home/jin_zd/.vscode-server/extensions/ms-vsco
1742 pts/0     Sl+        0:11 /home/jin_zd/.vscode-server/extensions/ms-vsco
19100 pts/5     Ss         0:00 /bin/bash --init-file /home/jin_zd/.vscode-ser
19222 pts/5     S+         0:00 /bin/bash ./tsh_output.sh
19524 pts/5     S+         0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s .
19525 pts/5     S+         0:00 ./tsh -p
19546 pts/0     S+         0:00 /bin/sh -c "/home/jin_zd/.vscode-server/bin/91
19547 pts/0     S+         0:00 /bin/bash /home/jin_zd/.vscode-server/bin/91fa
19555 pts/0     S+         0:00 sleep 1
19560 pts/5     R          0:00 /bin/ps a

140 tsh> fg %1
141 tsh> /bin/ps a
142  PID TTY          STAT       TIME COMMAND
143+ 25181 pts/3     S          0:00 -usr/local/bin/tcsh -i
144+ 26239 pts/3     S          0:00 make tshrefout
145+ 26240 pts/3     S          0:00 /bin/sh -c make tests > tshref.out 2>&1
146+ 26241 pts/3     S          0:00 make tests

147+ 26310 pts/3     S          0:00 perl ./sdriver.pl -t trace13.txt -s ./tsh -
148+ 26311 pts/3     S          0:00 ./tsh -p

149+ 26320 pts/3     R          0:00 /bin/ps a

```

测试结论：通过测试

4.14 测试用例 trace14.txt 的输出截图


```

./sdriver.pl -t trace14.txt -s ./tsh -a "-p"
#
# trace14.txt - Simple error handling
#
tsh> ./bogus
./bogus: Command not found.
tsh> ./myspin 4 &
[1] (19567) ./myspin 4 &
tsh> fg
fg command requires PID or %jobid argument
tsh> bg
bg command requires PID or %jobid argument
tsh> fg a
fg: argument must be a PID or %jobid
tsh> bg a
bg: argument must be a PID or %jobid
tsh> fg 9999999
(9999999): No such process
tsh> bg 9999999
(9999999): No such process
tsh> fg %2
%2: No such job
tsh> fg %1
Job [1] (19567) stopped by signal 20
tsh> bg %2
%2: No such job
tsh> bg %1
[1] (19567) ./myspin 4 &
tsh> jobs
[1] (19567) Running ./myspin 4 &

```

```

150 ./sdriver.pl -t trace14.txt -s ./tsh -a "-p"
151 #
152 # trace14.txt - Simple error handling
153 #
154 tsh> ./bogus
155+ ./bogus: Command not found
156 tsh> ./myspin 4 &
157+ [1] (26326) ./myspin 4 &
158 tsh> fg
159 fg command requires PID or %jobid argument
160 tsh> bg
161 bg command requires PID or %jobid argument
162 tsh> fg a
163 fg: argument must be a PID or %jobid
164 tsh> bg a
165 bg: argument must be a PID or %jobid
166 tsh> fg 9999999
167 (9999999): No such process
168 tsh> bg 9999999
169 (9999999): No such process
170 tsh> fg %2
171 %2: No such job
172 tsh> fg %1
173+ Job [1] (26326) stopped by signal 20
174 tsh> bg %2
175 %2: No such job
176 tsh> bg %1
177+ [1] (26326) ./myspin 4 &
178 tsh> jobs
179+ [1] (26326) Running ./myspin 4 &

```

测试结论：通过测试

4.15 测试用例 trace15.txt 的输出截图

```

./sdriver.pl -t trace15.txt -s ./tsh -a "-p"
#
# trace15.txt - Putting it all together
#
tsh> ./bogus
./bogus: Command not found.
tsh> ./myspin 10
Job [1] (19600) terminated by signal 2
tsh> ./myspin 3 &
[1] (19626) ./myspin 3 &
tsh> ./myspin 4 &
[2] (19628) ./myspin 4 &
tsh> jobs
[1] (19626) Running ./myspin 3 &
[2] (19628) Running ./myspin 4 &
tsh> fg %1
Job [1] (19626) stopped by signal 20
tsh> jobs
[1] (19626) Stopped ./myspin 3 &
[2] (19628) Running ./myspin 4 &
tsh> bg %3
%3: No such job
tsh> bg %1
[1] (19626) ./myspin 3 &
tsh> jobs
[1] (19626) Running ./myspin 3 &
[2] (19628) Running ./myspin 4 &
tsh> fg %1
tsh> quit

```

```

180 ./sdriver.pl -t trace15.txt -s ./tsh -a "-p"
181 #
182 # trace15.txt - Putting it all together
183 #
184 tsh> ./bogus
185+ ./bogus: Command not found
186 tsh> ./myspin 10
187+ Job [1] (26343) terminated by signal 2
188 tsh> ./myspin 3 &
189+ [1] (26345) ./myspin 3 &
190 tsh> ./myspin 4 &
191+ [2] (26347) ./myspin 4 &
192 tsh> jobs
193+ [1] (26345) Running ./myspin 3 &
194+ [2] (26347) Running ./myspin 4 &
195 tsh> fg %1
196+ Job [1] (26345) stopped by signal 20
197 tsh> jobs
198+ [1] (26345) Stopped ./myspin 3 &
199+ [2] (26347) Running ./myspin 4 &
200 tsh> bg %3
201 %3: No such job
202 tsh> bg %1
203+ [1] (26345) ./myspin 3 &
204 tsh> jobs
205+ [1] (26345) Running ./myspin 3 &
206+ [2] (26347) Running ./myspin 4 &
207 tsh> fg %1
208 tsh> quit

```

测试结论：通过测试

4.16 测试用例 trace16.txt 的输出截图

```

./sdriver.pl -t trace16.txt -s ./tsh -a "-p"
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
# signals that come from other processes instead of the terminal.
#
tsh> ./mystop 2
Job [1] (19665) stopped by signal 20
tsh> jobs
[1] (19665) Stopped ./mystop 2
tsh> ./myint 2
Job [2] (19675) terminated by signal 2

```

```

209 ./sdriver.pl -t trace16.txt -s ./tsh -a "-p"
210 #
211 # trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
212 # signals that come from other processes instead of the terminal.
213 #
214 tsh> ./mystop 2
215+ Job [1] (26359) stopped by signal 20
216 tsh> jobs
217+ [1] (26359) Stopped ./mystop 2
218 tsh> ./myint 2
219+ Job [2] (26362) terminated by signal 2
220+ make[1]: Leaving directory '/afs/cs.cmu.edu/project/ics/im/labs/shlab/s

```

测试结论：通过测试

5. 实验中遇到的问题及解决方法

(包括设计过程中的错误及测试过程中遇到的问题)

问题：在实现 eval 函数时，最初没有正确阻塞 SIGCHLD 信号，导致在添加作业到作业列表前，子进程可能已经终止并触发了 SIGCHLD 处理函数，但此时作业尚未添加到作业列表中。

解决：在 fork 子进程前阻塞 SIGCHLD 信号，添加作业后再解除阻塞

问题：在实现 waitfg 函数时，最初使用了忙等待循环，导致 CPU 资源浪费

解决：使用 sigsuspend 暂停 shell 进程直到收到信号，避免忙等待

问题：在实现子进程执行部分时，没有设置独立的进程组，导致在发送 SIGINT 或 SIGTSTP 信号时，错误地中断了 shell 本身

解决：在子进程中使用 setpgid(0, 0) 创建一个新的进程组

6. 请总结实验的收获，并给出对实验内容的建议

总结：通过完成这个 Shell 实验，我深刻理解了操作系统中进程控制、信号处理和作业管理的核心概念。实验过程中，最大的收获是对 Linux 系统编程中信号机制的理解更加深入。在编写信号处理函数如 sigchld_handler、sigint_handler 和 sigtstp_handler 时，我意识到信号处理的复杂性和重要性，特别是如何避免竞争条件和正确处理信号到达的时机。实验中通过 sigprocmask 函数阻塞和解除阻塞信号，这帮助我理解了如何在关键代码段执行期间防止信号干扰。

建议：我认为可以在实验设计中增加更多对错误情况的测试用例。例如，对于无效命令、非法参数、超长命令行等边界情况的处理，这些在实际系统中很常见。此外，实验可以增加一些高级功能的挑战性任务，比如实现输入/输出重定向、管道命令，或者命令历史记录功能，让学生能够更全面地理解 Shell 的工作原理。